metric_feature Â¶

This module implements the concrete `MetricFeature` that allows the manipulation of metrics in plans.

From an API perspective there are two properties that even though are persisted at the "metric level", they have to be defined and managed at the operator/plan level":

1. `filter_incomplete_instances` : When true, the output of the Metrics **operator** will exclude instances whose timestamp is older than the period defined by the start of the dataset and the largest window.
2. `timestamp_field` : Defined at the **plan** level in deployable plans.

AGG_DEP_TYPE_NAME `module-attribute` Â¶

```
AGG_DEP_TYPE_NAME = 'aggregation'
```

DEFAULT_FILTER_INCOMPLETE_INSTANCES `module-attribute` Â¶

```
DEFAULT_FILTER_INCOMPLETE_INSTANCES = False
```

DEFAULT_TIMESTAMP_FIELD `module-attribute` Â¶

```
DEFAULT_TIMESTAMP_FIELD = ''
```

FILTER_DEP_TYPE_NAME `module-attribute` Â¶

```
FILTER_DEP_TYPE_NAME = 'filter'
```

GROUP_BY_DEP_TYPE_NAME `module-attribute` Â¶

```
GROUP_BY_DEP_TYPE_NAME = 'group_by'
```

JOptional `module-attribute` Â¶

```
JOptional = autoclass('java.util.Optional')
```

LOGGER `module-attribute` Â¶

```
LOGGER = getLogger(__name__)
```

LOOKUP_DEP_TYPE_NAME `module-attribute` Â¶

```
LOOKUP_DEP_TYPE_NAME = 'lookup'
```

Long `module-attribute` Â¶

```
Long = autoclass('java.lang.Long')
```

METRIC_TS_DEP_TYPE_NAME module-attribute Â¶

```
METRIC_TS_DEP_TYPE_NAME = 'metric_ts'
```

MetricChannelConfig module-attribute Â¶

```
MetricChannelConfig = autoclass(MODEL_CLASSPATH +
'datascience.plan.operator.MetricChannelConfig')
```

MetricCoalesce module-attribute Â¶

```
MetricCoalesce = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.coalesce.MetricCoalesce')
```

MetricFeatureJava module-attribute Â¶

```
MetricFeatureJava = autoclass(DS_API_CLASSPATH + 'plan.features.MetricFeature')
```

UpdatePeriod module-attribute Â¶

```
UpdatePeriod = autoclass(MODEL_CLASSPATH + 'fraudclassifierdefinition.UpdatePeriod')
```

UpdatePeriodType module-attribute Â¶

```
UpdatePeriodType = autoclass(PULSE_API_CLASSPATH + 'apps.enums.UpdatePeriodType')
```

WindowConfig module-attribute Â¶

```
WindowConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.type.WindowConfig')
```

MetricFeature Â¶

```
MetricFeature(name: str, groupby_fields: list[str], aggregation: str, window_size: str, is_real_
time: bool = True, *, lookup_fields: list[str] | None = None, scheduled_cron_expression: str | N
one = None, filter_metric: str | None = None, delay: int | None = None, is_tumbling: bool =
False, is_infinite: bool = False, main_channel_contributes: bool = True, secondary_channels:
dict[str, MetricSecondaryChannelConfig] | None = None, cascaded_scheduled_metrics: bool = False,
coalesce: float | int | str | None = None, description: str | None = None)
```

Bases: BaseFeature

Represents a Metric feature in Pulse.

Parameters:

Name	Type	Description	Default
name Â¶	str	Name of the metric feature.	*required*
groupby_fields Â¶	list of str	Fields to group by.	*required*
	str		*required*

Name	Type	Description	Default
`aggregation` Â¶		Aggregation expression. Aggregations must be written in PQL: "avg","min","max","count","countDistinct","sum","stddev", "last","first","prev",... Fields must be enclosed with the pattern `${field}` eg. `'avg(${amount})'`, `'countDistinct(${MerchantCity})'`, `'count()'`. **Note that "advanced" aggregations are not compatible with Railgun**.	
`window_size` Â¶	`str`	Window expression. Typically, "1 day", "4 weeks", etc.	*required*
`is_real_time` Â¶	`bool`	True = Real time window, False = Scheduled window. True by default.	`True`
`scheduled_cron_expression` Â¶	`str`	If is_real_time = False, a cron expression must be specified to define the periodicity of the window, by default None e.g. `'0 0 0 * * ?'` for a scheduled window running every day at midnight.	`None`
`filter_metric` Â¶	`str`	Filter expression, by default None. Fields must be enclosed with the pattern `${field}` eg. `"${TransactionAmount} > 100 and ${Fraud} != false"`.	`None`
`delay` Â¶	`int`	Defines delay in milliseconds, must be greater than 1000, by default None.	`None`
`is_tumbling` Â¶	`bool`	Tumbling window, by default False.	`False`
`is_infinite` Â¶	`bool`	When True, this metric will be considered a "non-windowed" metric. *Requires Pulse 25.0.89.* **Experimental Feature.**	`False`
`description` Â¶	`str`	Metric description, by default None.	`None`
`lookup_fields` Â¶	`list of str`	Lookup fields, ie the fields used to query the aggregation. By default these are set to be the same as the `groupby_fields`. When different, it means that one can execute a query like: `group by sender -> lookup by receiver`. *Experimental feature.*	`None`
	`bool`	When false, the main channel does not contribute to the aggregation. True by default. *Omnichannel feature*.	`True`

Name	Type	Description	Default
`main_channel_contributes` Â¶			
`secondary_channels` Â¶	`dict[str, MetricSecondaryChannelConfig]`	Mapping of channel ID to a `MetricSecondaryChannelConfig`. Empty by default, ie no secondary channels configured. *Omnichannel feature*.	`None`
`cascaded_scheduled_metrics` Â¶	`bool`	When True, it indicates this feature is in a plan with cascading scheduled metrics that depend on each other. False by default.	`False`
`coalesce` Â¶	`float or int or str`	Coalesce value for when the metric aggregation is empty.	`None`

Raises:

Type	Description
`ValueError`	If is_real_time = False and scheduled_cron_expression is None. If the timestamp field from the schema is not specified in the Plan. If delay is less than 1000 ms.

aggregation `property` `writable` Â¶

```
aggregation
```

cascaded_scheduled_metrics `property` `writable` Â¶

```
cascaded_scheduled_metrics: bool
```

coalesce `property` `writable` Â¶

```
coalesce: float | int | str | None
```

delay `property` `writable` Â¶

```
delay
```

description `property` `writable` Â¶

```
description
```

filter_incomplete_instances `property` `writable` Â¶

```
filter_incomplete_instances: bool
```

filter_metric property writable Â¶

```
filter_metric
```

groupby_fields property writable Â¶

```
groupby_fields
```

is_infinite property writable Â¶

```
is_infinite: bool
```

is_real_time property writable Â¶

```
is_real_time
```

is_tumbling property writable Â¶

```
is_tumbling: bool
```

lookup_fields property writable Â¶

```
lookup_fields
```

main_channel_contributes property writable Â¶

```
main_channel_contributes: bool
```

When False, the main channel will not contribute to the metric.

Omnichannel

This property is part of the Omnichannel suite of properties.

name property writable Â¶

```
name
```

operator property Â¶

```
operator
```

scheduled_cron_expression property writable Â¶

```
scheduled_cron_expression
```

secondary_channels property writable Â¶

```
secondary_channels: dict[str, MetricSecondaryChannelConfig]
```

Mapping between secondary channels and their configuration.

Omnichannel

This property is part of the Omnichannel suite of properties.

timestamp_field property writable Â¶

```
timestamp_field: str
```

window_size `property` `writable` Â¶

```
window_size
```

auto_rename Â¶

```
auto_rename()
```

Automatically renames the metric based on its configuration.

Default naming: `lowercase({groupby_fields}_{aggregation}_{aggregation fields}_{window size}_{is_real_time})` eg. accountid_avg_amount_1h_rt. Advanced aggregations will be named: `lowercase({groupby_fields}_agg_{window size}_{is_real_time})` .

from_dict `classmethod` Â¶

```
from_dict(feat_dict: dict[str, str]) -> MetricFeature
```

from_java_feature `classmethod` Â¶

```
from_java_feature(feature: JavaClass, operator: BaseFeaturesOperator) -> MetricFeature
```

Creates a MetricFeature from a Java DS-API MetricFeature.

Parameters:

Name	Type	Description	Default
`feature` Â¶	`Java DS-API MetricFeature`		*required*
`operator` Â¶	`MetricOperator`		*required*

Returns:

Type	Description
`MetricFeature`	MetricFeature

get_default_naming Â¶

```
get_default_naming() -> str
```

Returns a default name based on its configuration.

Default naming: `lowercase({groupby_fields}_{aggregation}_{aggregation fields}_{window size}_{is_real_time})` eg. accountid_avg_amount_1h_rt. Advanced aggregations will be named: `lowercase({groupby_fields}_agg_{window size}_{is_real_time})`

In the case of "advanced" window expressions, the window expression is omitted from the name.

Railgun compatibility

Note that "advanced" window expressions are not compatible with Railgun. Please use only the appropriate set of window expressions.

Returns:

Type	Description
`str`	Default name

get_dependencies Â¶

```
get_dependencies() -> set[str]
```

get_dependencies_by_type Â¶

```
get_dependencies_by_type() -> dict[str, list[str]]
```

rename_dependencies Â¶

```
rename_dependencies(old_name: str, new_name: str)
```

to_dict Â¶

```
to_dict(with_operator: bool = True) -> dict[str, str | float | int | bool | None]
```

to_java_feature_builder Â¶

```
to_java_feature_builder() -> JavaClass
```

From MetricFeature to a Pulse API MetricOperatorConfig.

Returns:

Type	Description
`Builder`	MetricOperatorConfig.Builder

MetricSecondaryChannelConfig `dataclass` Â¶

```
MetricSecondaryChannelConfig(groupby_fields: list[str], aggregation_fields: list[str], filter_ex
pr: str | None = None)
```

Pythonic representation of a MetricChannelConfig.

Info

Using this dataclass is required to interact with Omnichannel plans.

aggregation_fields `instance-attribute` Â¶

```
aggregation_fields: list[str]
```

filter_expr `class-attribute` `instance-attribute` Â¶

```
filter_expr: str | None = None
```

groupby_fields `instance-attribute` Â¶

```
groupby_fields: list[str]
```

from_java `classmethod` Â¶

```
from_java(cfg: JavaClass) -> MetricSecondaryChannelConfig
```

Return the Python representation of this channel from its Java counterpart.

to_java Â¶

```
to_java() -> JavaClass
```

Return the Java representation of this channel configuration.